

REWINDCAST

A Step-by-Step Beginner's Guide to Building a Full-Stack AI Product

RewindCast is a commercial-grade "Previously On..." spoken recap engine designed for podcasts and long-form audio. This guide takes you from an absolute beginner to understanding the full-stack architecture, API frameworks, AI model integrations, and web component engineering used to build it.

ENGINEERED BY ANTIGRAVITY AI

May 2026 Edition

Target Stack: Python FastAPI + Next.js 14 + Gemini API

Chapter 1: Architecture & Technology Stack

To build a production-ready AI application, we choose technologies that balance performance, simplicity, and scalability. RewindCast is built as a decoupled full-stack application containing a robust FastAPI backend and a sleek Next.js 14 frontend.

Why This Stack Was Chosen:

- * **FastAPI (Python):** FastAPI is the standard for high-performance AI APIs. Because AI libraries (like Whisper, PyTorch, and Google GenAI) are natively built in Python, FastAPI is the perfect bridge, providing async endpoints, auto-generated documentation, and rapid request execution.
- * **Next.js 14 + TypeScript:** Provides a world-class framework for rendering highly interactive and visually spectacular interfaces. It handles fast client-side navigation, component separation, and serves as a robust interface for media player control.
- * **faster-whisper:** Instead of calling external paid APIs, we run transcription locally using the highly optimized faster-whisper library (a C++ reimplementation of OpenAI's Whisper). It is up to 4x faster than standard Whisper and has a memory footprint under 500MB on a CPU.
- * **Google Gemini API (gemini-2.5-flash):** Acts as our central reasoning engine. It takes the text segments within the listened timestamp range and synthesizes them into an engaging narrative while keeping costs under fractions of a penny.
- * **Edge TTS (edge-tts):** A python wrapper that streams audio from Microsoft Edge's translation servers. It provides exceptionally high-quality, natural-sounding, human-like voice narration entirely for free.

Chapter 2: Workspace & Foundation Setup

We start by organizing our codebase. Decoupling the codebase into 'backend/' and 'frontend/' folders keeps the dependencies isolated and standardizes project organization.

1. Create Directory Structure:

Initialize your workspace and create the primary directories for the backend, frontend, data cache, and technical documents:

```
mkdir rewindcast && cd rewindcast
mkdir -p backend/app/routers backend/app/services backend/media backend/recaps
mkdir -p frontend/components frontend/lib frontend/app
mkdir -p docs
```

2. Environmental Configuration (.env):

To securely configure API keys and media directories without hardcoding them, we use an environmental file (.env). We copy this into the project root and ensure it is added to .gitignore.

```
# .env - Configuration Environment
GEMINI_API_KEY=AIZA...
MEDIA_DIR=./media
RECAPS_DIR=./recaps
TTS_VOICE=en-US-AriaNeural
WHISPER_MODEL_SIZE=base
```

3. Git Integration (.gitignore):

To prevent large media files, compiled virtual environments, databases, and private API keys from being tracked or accidentally exposed on GitHub, we add them to .gitignore:

```
# Python virtual environments
.venv/

# Environment variables containing keys
.env

# Local directories caching media and transcripts
backend/media/*
backend/recaps/*
frontend/node_modules/
*.db
```

Chapter 3: Engineering the Backend Core

The backend serves as our operational engine, handling database caching, audio transcription, AI summarization, and speech-to-text generation.

1. Database & Schema Configuration:

Instead of transcribing the same file every time a user requests a recap, we use SQLite along with SQLAlchemy to cache transcripts. The `db_models.py` defines three primary entities:

- * **Media:** Stores the title, unique filename on disk, and duration of the imported podcast.
- * **Transcript:** Stores the full text and a JSON string of word-by-word segments with start/end times.
- * **Recap:** Caches generated recap text, bullet points, and the file path of the generated TTS MP3.

2. Intelligent Text Chunker:

When summarizing long podcasts (e.g., a 1-hour conversation), the text is too massive to pass directly. We wrote a smart chunker in `services/chunker.py` that splits the transcript into 1500-token blocks with a 15% overlap. This preserves continuity between adjacent blocks.

3. Google Gemini Orchestration:

To make summaries highly cohesive, we implement a Map-Reduce strategy using Gemini:

- Map Phase: Each text chunk is summarized concurrently into a concise paragraph.
- Reduce Phase: All paragraph summaries are combined and Gemini synthesizes them into a single 'Previously On...' script using the present tense and narrator-style tone.

The model is instructed to output a structured JSON containing the text and exact bullet points.

4. Spoken Narrator (TTS):

Once Gemini generates the script, we stream it into `edge-tts` to write a natural-sounding, expressive narrator voice as an MP3 file, creating a professional radio-show feel.

5. Dynamic Duration Scaling (NEW):

To make the product commercially viable, the recap length dynamically adjusts to match the amount of content a user has already listened to. A 50-second listen yields a 10s recap, whereas a 1-hour listen yields a highly polished 1.5-minute summary.

```
if listened_seconds <= 60:
    return 30_words, 3_points # ~10 seconds audio
elif listened_seconds <= 300:
    return 50_words, 3_points # ~20 seconds audio
elif listened_seconds <= 3600:
    return 180_words, 5_points # ~1.5 mins audio
```

Chapter 4: Engineering the Next.js Frontend

The frontend translates technical API capability into an intuitive, responsive, and visually exceptional user interface using Next.js client-side components.

1. Dynamic Theme & Aesthetics (globals.css):

We styled the application using pure vanilla CSS variables to ensure perfect layout control. The design system incorporates modern, dark-mode glassmorphism:

- Color Palette: Deep rich obsidian, high-contrast white text, and glowing neon purple accents.
- Blurring & Translucency: Uses backdrop-filters (e.g. backdrop-filter: blur(16px)) to create glowing layers that hover over other visual elements.
- Hover States: Glowing borders, smooth scale transformations, and interactive color shifts.

2. Custom Waveform Player (MediaPlayer.tsx):

Instead of a browser-default audio player, we built a fully responsive player from scratch. It includes a decorative soundwave visualizer of 40 bars that animates in sync with the audio duration and playback progress. It also supports multiple listening speeds (0.75x to 2x) and automatically caches playback progress in localStorage, prompting you to resume if you refresh.

3. Glowing Recap Trigger (RecapButton.tsx):

The primary product feature button is styled with a subtle pulse animation. It dynamically calculates the estimated recap duration, and displays a multi-phase loading sequence ('Transcribing...' -> 'Summarizing...' -> 'Synthesizing voice...') during the async generation process to keep users engaged.

4. YouTube Ingest with Custom Time Selector:

To allow users to request summaries of YouTube videos immediately, we built a powerful timestamp options panel. Users check 'Auto-generate recap up to a custom timestamp', input the minutes and seconds (e.g., 25 mins), and click submit. The frontend captures this input, converts it to seconds, and redirects the user with a recap_at query parameter.

```
// In page.tsx: Detect query parameters on mount
useEffect(() => {
  const params = new URLSearchParams(window.location.search);
  const recapAt = params.get('recap_at');
  if (recapAt) setAutoTriggerTime(parseFloat(recapAt));
}, []);
```

Chapter 5: Deploying & Launching

To run the application or package it as a commercial product, we configure both local run commands and Docker containers.

1. Run Locally:

To test or develop locally on your machine, activate the Python virtual environment for the backend, install the dependencies, and start uvicorn. In another terminal, install frontend dependencies and start the Next.js dev server:

```
# Terminal 1: Backend
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -e .
uvicorn app.main:app --reload --port 8000

# Terminal 2: Frontend
cd frontend
npm install
npm run dev
```

2. One-Command Docker Launch:

For seamless containerized deployment, we added a multi-service docker-compose.yml file. It sets up a shared network, maps backend storage directories, and starts both services in production:

```
docker-compose up --build
```

3. Productization & Pitching:

If you want to present this as a product to platforms like Youtube, Spotify, or venture capitalists:

- Reference our docs/PRODUCT_BRIEF.md: It includes complete customer journey plans, market size data, B2B SaaS licensing fee structures, and detailed operating costs.
- Emphasize operating margins: With local Whisper transcription and Microsoft Edge TTS, your ONLY active operating cost is Gemini API tokens (~\$0.002 per recap). This represents a 99.5% gross margin, making the product highly lucrative at scale.

Congratulations! You are now ready to dominate the AI Audio Space.